تجميع الكائنات (Grouping Objects)

1 مصفوفة الكائنات — Array of Objects

الفكرة ببساطة:

تماماً كما تصنع مصفوفة من الأرقام `int arr[5]`، يمكنك صنع مصفوفة كاملة من **الكائنات**. كل خانة في المصفوفة هي كائن مستقل يملك بياناته الخاصة

```cpp
class employee
{
private:
    char name[20];
    int age, sal;
public:
    void getdata();
    void putdata();
};

int main()
{
    employee emp[5];        // مصفوفة من 5 كائنات

    for(int i = 0; i < 5; i++)
        emp[i].getdata();   // تدخل البيانات لكل كائن

    for(int i = 0; i < 5; i++)
        emp[i].putdata();   // تطبع بيانات كل كائن
}
```

> 📌 القاعدة: للوصول لدوال كائن معين في المصفوفة نستخدم: `emp[i].functionName()`
> النقطة تبقى كما هي، فقط نضع `[i]` بعد اسم المصفوفة.

متى نستخدم مصفوفة الكائنات؟

عندما يكون لديك عدد من الكيانات المتشابهة — مثل 5 موظفين، 10 طلاب، 20 منتج. بدلاً من كتابة `emp1`, `emp2`, `emp3` ... تكتب `emp[0]`, `emp[1]`, `emp[2]` وتتعامل معها بحلقة

مثال: مصفوفتان في نفس الوقت

```cpp
int main()
{
    emp foreman[5];     // مصفوفة لـ 5 رؤساء عمال
    emp engineer[5];    // مصفوفة لـ 5 مهندسين

    for(int i = 0; i < 5; i++)
        foreman[i].getdata();   // إدخال بيانات الرؤساء

    for(int i = 0; i < 5; i++)
        engineer[i].getdata();  // إدخال بيانات المهندسين
}
```

> ⚠️ فخ امتحاني: كل مصفوفة مستقلة تماماً — التغيير في `foreman[0]` لا يؤثر أبداً على `engineer[0]`.

سؤال MCQ متوقع (1):

> ما الصيغة الصحيحة لاستدعاء دالة `getdata()` للكائن الثالث في مصفوفة `emp`؟
>
> - أ) `emp.getdata(3)`
> - ب) `emp(3).getdata()`
> - ج) ✅ `emp[2].getdata()`
> - د) `getdata(emp[3])`

2 المصفوفة كعضو داخل الكلاس — Array of Members

الفكرة ببساطة:

هنا العكس تماماً — المصفوفة ليست **من** كائنات، بل موجودة **داخل** الكلاس كعضو بيانات. الكلاس نفسه يحتوي على مصفوفة

```cpp
const int m = 50;

class items
{
    int   item_code[m];     // مصفوفة داخل الكلاس
    float item_price[m];    // مصفوفة داخل الكلاس
    int   count;            // عداد للعناصر
public:
    void cnt()            { count = 0; }
    void get_item();
    void display_sum();
    void display_item();
};

void items::get_item()
{
    cout << "enter item code: ";
    cin  >> item_code[count];   // نخزن في الخانة الحالية
    cout << "enter item cost: ";
    cin  >> item_price[count];
    count++;                    // نقدّم العداد للخانة التالية
}

void items::display_sum()
{
    float sum = 0;
    for(int i = 0; i < count; i++)
        sum = sum + item_price[i];
    cout << "total value: " << sum;
}
```

> 📌 القاعدة: المصفوفة داخل الكلاس تعمل مثل أي عضو عادي — لكن تنتبه لتهيئة العداد `count=0` قبل الاستخدام، لأن عدم تهيئته يعطي قيمة عشوائية ويُفسد التخزين.

جدول المقارنة — الفرق بين النوعين

العنصر	مصفوفة الكائنات	المصفوفة كعضو في الكلاس
أين المصفوفة؟	خارج الكلاس في الـ `main`	داخل الكلاس كعضو
ما نوع محتواها؟	كائنات من الكلاس	بيانات عادية (int, float...)
مثال التعريف	`;employee emp[5]`	`;int item_code[50]` داخل الكلاس
طريقة الوصول	`()emp[i].getdata`	مباشرة داخل دوال الكلاس

```
class sort{
private:
    int nm[30];     // المصفوفة خاصة — محمية من الخارج
public:
    void getdata();  // تُدخل الأرقام وتُرتّبها
    void putdata();  // تطبع الأرقام المرتبة
};

void sort::getdata(){
    int i, j, k;
    cout << "enter 10 numbers: ";
    for(i = 0; i < 10; i++)
        cin >> nm[i];
    // Bubble Sort — الترتيب التصاعدي
    for(i = 0; i < 9; i++)
        for(j = i+1; j < 10; j++)
            if(nm[i] > nm[j]){
                k      = nm[i];  // تبادل
                nm[i] = nm[j];
                nm[j] = k;
            }
}

int main(){
    sort s;
    s.getdata();    // إدخال + ترتيب
    s.putdata();    // طباعة
}
```

4 برنامج القائمة بالـ switch — Menu-Driven Program

هذا نمط برمجي مهم جداً في الامتحانات. الفكرة هي استخدام `do-while` مع `switch` لإعطاء المستخدم خيارات متعددة

```cpp
int main()
{
    items order;
    order.cnt();       // تهيئة العداد أولاً
    int x;

    do {
        cout << "1: Add item" << endl;
        cout << "2: Display total" << endl;
        cout << "3: Remove item" << endl;
        cout << "4: Display all" << endl;
        cout << "5: Quit" << endl;
        cin >> x;

        switch(x)
        {
            case 1: order.get_item();    break;
            case 2: order.display_sum(); break;
            case 4: order.display_item();break;
            case 5: break;
            default: cout << "Error! Try again";
        }
    } while(x != 5);    // يكرر حتى يختار المستخدم 5
}
```

> 📌 لماذا `do-while` وليس `while` عادية؟ لأن `do-while` تُنفِّذ جسم الحلقة مرة واحدة على الأقل قبل الفحص — وهذا منطقي هنا لأننا نريد عرض القائمة أولاً ثم السؤال.

📊 جدول المقارنة الشامل — ملخص الوحدة

المفهوم	التعريف	الصيغة	متى نستخدمه؟
مصفوفة الكائنات	مصفوفة كائناتها من نفس الكلاس	`ClassName arr[n]`	عند تكرار نفس الكيان
المصفوفة كعضو	مصفوفة بيانات داخل الكلاس	`int arr[n]` داخل الكلاس	لتخزين قائمة بيانات
الترتيب بالكلاس	إخفاء خوارزمية الترتيب داخل دالة	`void sort::getdata()`	لتبسيط الواجهة
برنامج القائمة	`do-while + switch`	`do{...}while(x!=5)`	لخيارات متعددة للمستخدم